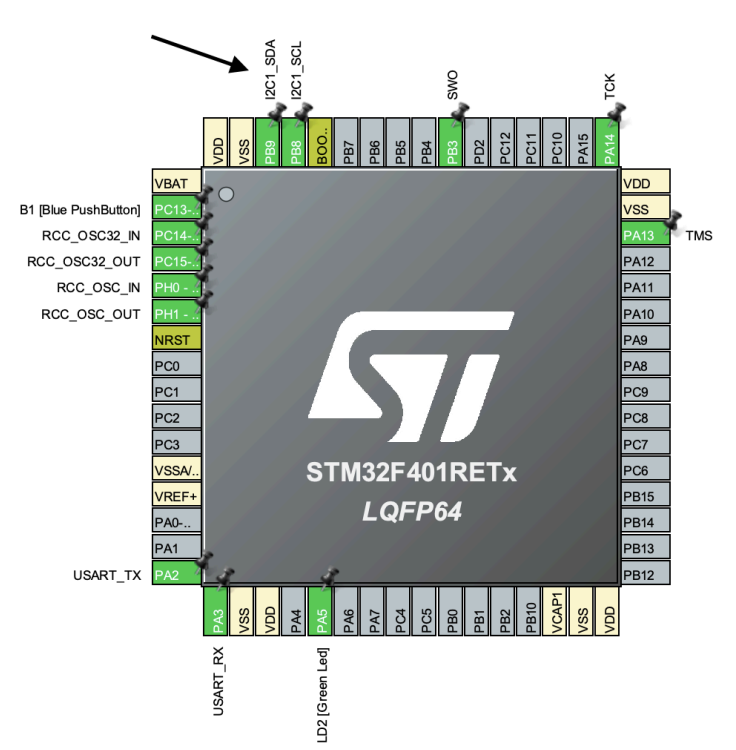
Mark	8/11

Team name:	B9		
Homework number:	Homework 08		
Due date:	11/11/2025		
Contribution:	No	Partial	Full
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

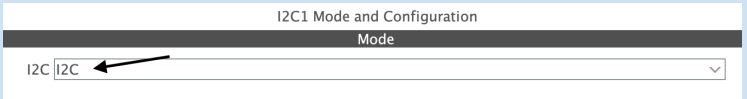
Project name	Accelerometer		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			✅

Part 1. Complete the MEMS accelerometer project 1b.

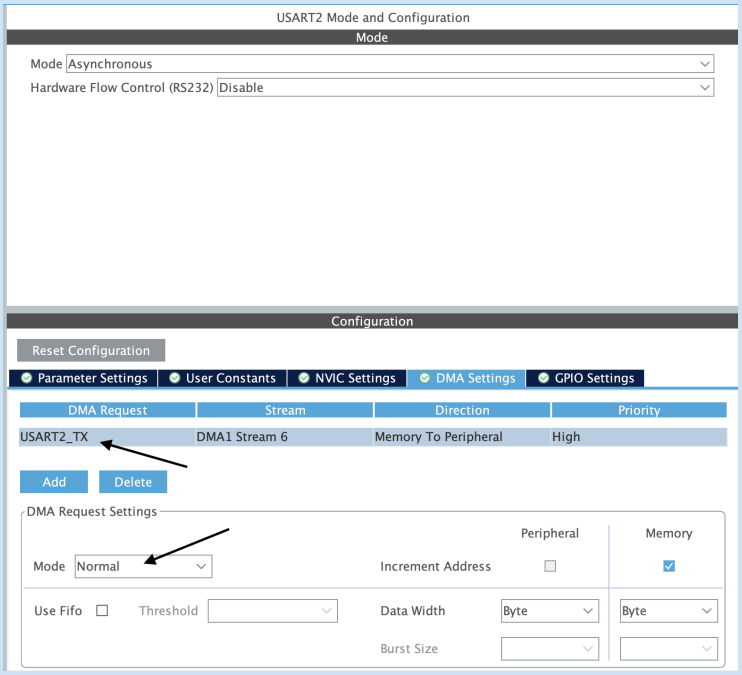
As in the previous homework we first set up the correct pins for I2C communication in the ioc configurator.

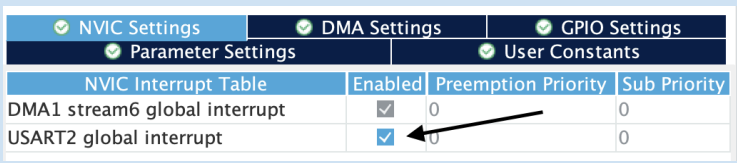


We then go into the connectivity section to enable the I2C1 (leaving everything else as default).

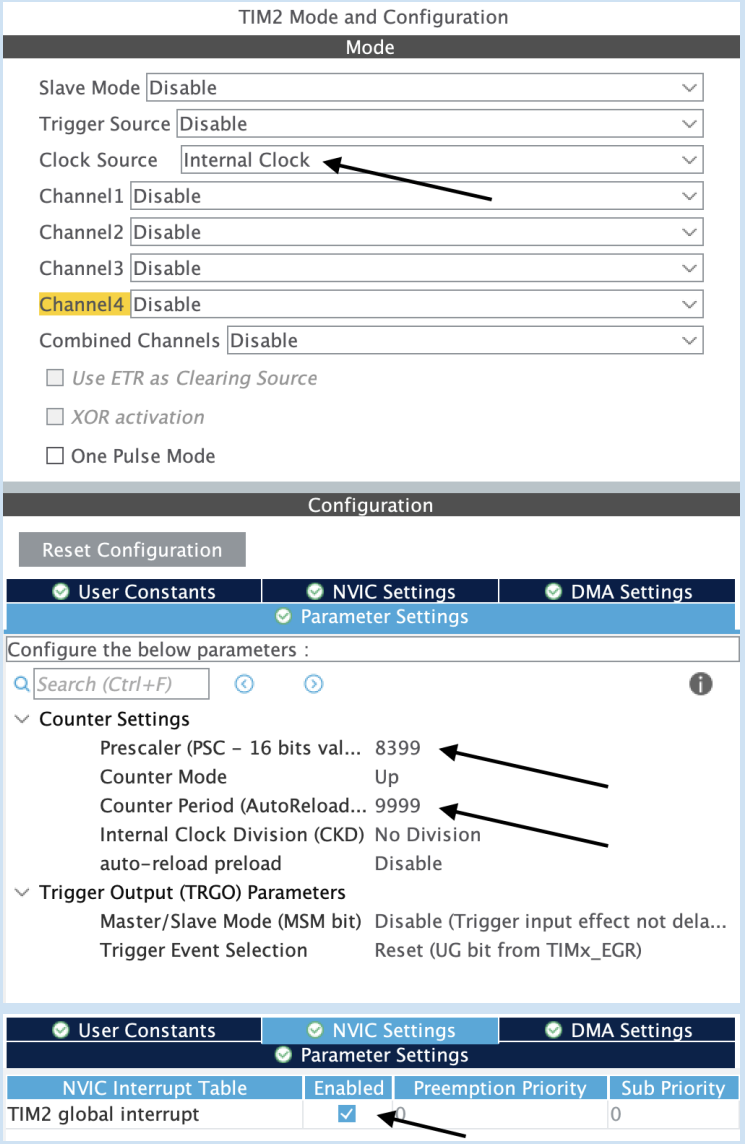


In the connectivity section we also enable the USART2 and we enable the interrupt and the DMA in transmission (as done in previous homeworks).





Now to trigger the reading every second we have to configure TIM2 and its global interrupt. We set the clock source, the prescaler at 8399, the counter at 9999 and enable the global interrupt.



We now go into the “main.c” file. In the main function we setup the accelerometer and start the timer.

```
/* USER CODE BEGIN 2 */
uint8_t i2c_buf[2];

i2c_buf[0] = CTRL_REG1;
i2c_buf[1] = 0x17; // ODR=1Hz (0001), Normal mode (0), ZEN, YEN, XEN (0111)
HAL_I2C_Master_Transmit(&hi2c1, LIS2DE_ADDR, i2c_buf, 2, 100);

i2c_buf[0] = CTRL_REG4;
i2c_buf[1] = 0x00; // FS=±2g (00)
HAL_I2C_Master_Transmit(&hi2c1, LIS2DE_ADDR, i2c_buf, 2, 100);

HAL_TIM_Base_Start_IT(&htim2);
```

Then we just need to write the callback for the timer. We first transmit to the accelerometer that we want to read from register 0x28 with autoincrement enabled (done with variable

start_reg), then receive 6 bytes, convert them to g values and send them to the terminal as usual.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM2) {
      HAL_I2C_Master_Transmit(&hi2c1, LIS2DE_ADDR, &start_reg, 1, 100);

      HAL_I2C_Master_Receive(&hi2c1, LIS2DE_ADDR, rx_buf, 6, 100);

      int8_t raw_x = (int8_t)rx_buf[1];
      int8_t raw_y = (int8_t)rx_buf[3];
      int8_t raw_z = (int8_t)rx_buf[5];

      float x_g = (float)raw_x * 0.0156f;
      float y_g = (float)raw_y * 0.0156f;
      float z_g = (float)raw_z * 0.0156f;

      char string[64];
      int length = snprintf(string, sizeof(string), "X: %+.2f g\r\nY: %+.2f g\r\nZ: %+.2f g\r\n\r\n", x_g, y_g, z_g);
      HAL_UART_Transmit_DMA(&huart2, (uint8_t *)string, length);
  }
}
```

To get it working we need to include the “stdio.h” library for the snprintf, enable the float in the printf, define all the addresses and variables used.

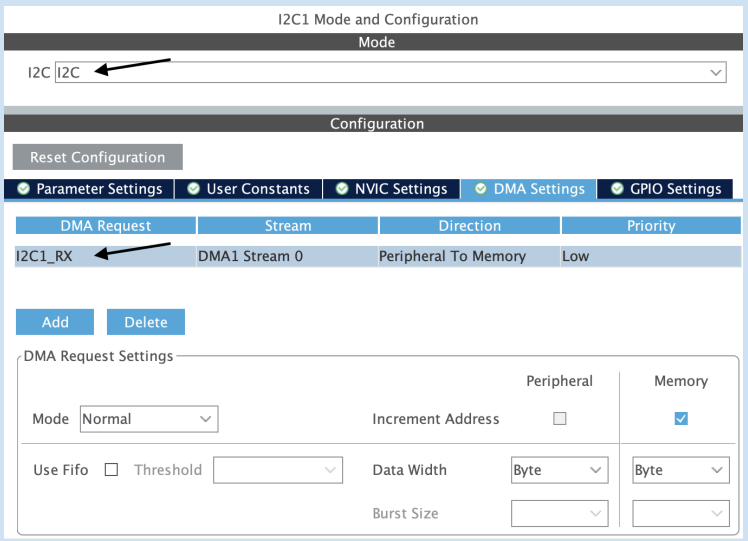
```
/* USER CODE BEGIN PD */
#define LIS2DE_ADDR (0b0101000 << 1)
#define CTRL_REG1 0x20
#define CTRL_REG4 0x23
#define OUT 0x28
/* USER CODE END PD */
```

```
/* USER CODE BEGIN PV */
uint8_t rx_buf[6];
uint8_t start_reg = OUT | 0b10000000;
/* USER CODE END PV */
```

We tested it and it seems to be working correctly.

Part 2. Complete the MEMS accelerometer project 1c.

In addition to everything we have done before, we now go into the connectivity section to enable both the DMA to receive and the interrupts (required to be enabled as said in the documentation for the DMA).



Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
DMA1 stream0 global interrupt	☑	0	0
I2C1 event interrupt	☑	0	0
I2C1 error interrupt	☑	0	0

In the “main.c” file everything stays the same except for the timer callback function. We modify it as follows.

```
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  if (htim->Instance == TIM2) {
      HAL_I2C_Master_Transmit(&hi2c1, LIS2DE_ADDR, &start_reg, 1, 100);

      HAL_I2C_Master_Receive_DMA(&hi2c1, LIS2DE_ADDR, rx_buf, 6);
  }
}

void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c) {
  if (hi2c->Instance == I2C1) {
      int8_t raw_x = (int8_t)rx_buf[1];
      int8_t raw_y = (int8_t)rx_buf[3];
      int8_t raw_z = (int8_t)rx_buf[5];

      float x_g = (float)raw_x * 0.0156f;
      float y_g = (float)raw_y * 0.0156f;
      float z_g = (float)raw_z * 0.0156f;

      char string[64];
      int length = snprintf(string, sizeof(string), "X: %+.2f g\r\nY: %+.2f g\r\nZ: %+.2f g\r\n\r\n", x_g, y_g, z_g);
      HAL_UART_Transmit_DMA(&huart2, (uint8_t *)string, length);
  }
}
/* USER CODE END 0 */
```

In the timer callback we just start the DMA receive. Then when it is finished we convert the value in the receive complete callback and send them to the terminal. We used the normal blocking function when transmitting to ensure completion before starting receiving.

We have tested it and it works.

Professor comments:	